btt006_unit_9_lab-solution

May 19, 2026

1 Lab 9: Building an Agentic Pitch Clinic with Routing, Evaluator-Optimizer, and MCP

[1]:
```
import openai_setup

from pydantic import BaseModel
from typing import Literal
from agents import Agent, Runner, set_tracing_disabled
from agents.mcp import MCPServerStdio
from IPython.display import display, Markdown

set_tracing_disabled(True)
```

Throughout this unit, you have explored what makes a system "agentic," practiced the core building blocks of AI agents (model, instructions, tools, memory, structured output), and built each of the named agentic workflow patterns: prompt chaining, routing, parallelization, orchestrator-worker, and evaluator-optimizer. You have also built your own Model Context Protocol (MCP) server and connected it to an agent.

In this lab, you will combine three of those skills into a single multi-agent system: a routing classifier on top, an evaluator-optimizer loop in the middle, and an MCP server underneath that grounds the evaluator in the company's own playbook.

You will complete the following tasks:

1. Set up the Pitchwise Playbook MCP server:
   - Fill in a `@mcp.tool()` function that returns a scoring rubric for a given pitch type
   - Write the server out to a file and confirm it exposes the expected tool
2. Build the classifier agent for routing:
   - Define a Pydantic schema that constrains the classifier's output to one of three pitch types
   - Write the classifier agent's instructions and run it on a couple of example pitches
3. Build a coach agent:
   - Read two coach agents that are provided for you
   - Write the third coach agent to match the pattern, and run it on an example draft
4. Build the evaluation pieces:
   - Define a Pydantic schema for the evaluator's structured per-criterion feedback
   - Write the instructions string the evaluator will use

- Confirm the evaluator can pull a rubric from the MCP server and return structured feedback

5. Wire the routing and evaluator-optimizer loop:
   - Complete the pipeline function: the loop body is the only part you write
   - Run the full pipeline on three founder pitches and observe what happens
6. Analysis:
   - Reflect on what the system did with each pitch and how you would explain the design to a non-technical partner
7. AI Reflection:
   - Reflect on how you used (or did not use) AI tools during this lab

Important: Most graded cells that require you to enter code will contain regions marked with # YOUR CODE HERE and # END OF YOUR CODE. Between these lines, you'll find the line raise NotImplementedError("Your code is missing."), like so:

```
# YOUR CODE HERE
raise NotImplementedError("Your code is missing.")
# END OF YOUR CODE
```

These markers indicate exactly where you should enter your answer. Replace the line raise NotImplementedError("Your code is missing.") with your code.

1.1 Business Context

Read through the scenario below. You will be putting yourself in the shoes of a junior ML engineer (MLE) at Pitchwise, a small startup accelerator where you have been asked to prototype an agentic pitch coaching system the partners can use to triage and improve founder pitches in the weeks before demo day.

1. Company and Context Pitchwise is a small startup accelerator based in Pittsburgh, founded in 2019 by two former operators. One of them built and sold a consumer hardware company, the other had a first startup that ran out of money before it found a market. They like to say that between them they have been to the moon and they have been to the floor, and that is exactly why founders trust them.

The accelerator runs two cohorts a year of about fifteen companies each. Pitchwise's whole pitch (no pun intended) is that the founders running it have actually sat on both sides of the table. They are opinionated about what makes a pitch land and what makes a pitch flop, and they have written those opinions down in a playbook that nobody outside the firm has ever seen.

The eight weeks leading up to demo day are the most intense part of the program. Every founder in the cohort goes through dozens of practice pitch sessions in front of the partners, who do not pull punches. One of the partners is known for closing her laptop ten seconds into a bad opening line and saying "try again." The other has a phrase he uses so often the cohort prints it on T-shirts: "I do not understand what your company does and I have been listening for two minutes." Founders dread these sessions and they love them, because the bar at Pitchwise is high and the partners' instincts are real.

2. Business Challenge This year's cohort is bigger than usual, and the partners are drowning. Drafts are piling up faster than the partners can review them, and feedback quality is starting to

slip because the partners are rushing through review sessions. The team has been asked whether an AI-assisted system could handle the first pass on pitch drafts, so the partners can focus their time on the second-pass review where their judgment actually adds value.

The partners are skeptical of generic AI coaching tools. They have tried two off-the-shelf products and were unimpressed: the feedback was generic and did not reflect what they actually care about. They want a system that is grounded in the Pitchwise playbook (their own rubric, their own standards, their own opinions) and not in whatever the model picked up from the open internet.

3. Business Goal The goal is to build a prototype agentic system that can take a rough draft of a founder's pitch and return a polished version. The system should:

- Recognize what kind of company is pitching. The partners have three loose categories: consumer products (sold directly to individual people), B2B SaaS (software sold to other businesses on a subscription), and deep tech (companies built around a specific piece of novel technology or science). Each category gets evaluated against a different rubric.
- Iterate on the pitch, scoring it against the matched rubric and revising it until it either meets the bar or has stopped improving.
- Ground its evaluation in the actual Pitchwise playbook, not in the model's own opinions about pitches.

If the prototype works, the partners will use it as a first-pass screen so they can spend their review time on the pitches that have already reached a baseline quality.

4. Your Role and Task You have just joined Pitchwise as a junior MLE on a two-person engineering team. The partners have given you their playbook (the three rubrics, one per pitch type) and three sample founder pitches that came in this week. Your job is to build the prototype and run it end-to-end on those three pitches so the partners can see what the system actually does.

This work requires judgment at every stage. The classifier prompt determines whether the right rubric gets pulled. The evaluator's instructions determine whether the model takes the playbook seriously or talks around it. The iteration cap determines whether you waste API calls on pitches that are not going to converge. The decisions you make about how to combine these pieces will determine whether the partners trust the system enough to use it.

5. Technical Focus in This Lab This lab focuses on combining three agentic patterns into a single system:

- **Routing**: Using a classifier agent with a constrained Pydantic output to dispatch the pitch to the matched downstream coach.
- **Evaluator-Optimizer**: Running a generate-evaluate-revise loop with structured per-criterion feedback and a hard iteration cap.
- **MCP**: Writing your own MCP server, connecting an agent to it via `MCPServerStdio`, and letting the evaluator pull grounding data (the rubric) from the server at run time.

1.2 Part 1. Set Up the Pitchwise Playbook MCP Server

The Pitchwise partners have given you their playbook in writing: three rubrics, one for each pitch type, that capture the three things they care most about when evaluating that kind of company.

You will expose this playbook as an MCP server so the evaluator agent can pull the matched rubric at runtime instead of carrying the playbook in its prompt.

You wrote MCP servers from scratch in the Implement a Model Context Protocol (MCP) Server assignment. The pattern here is the same: `FastMCP`, `@mcp.tool()`, `mcp.run()`.

Task: Complete the body of the `get_rubric` tool function inside the server code string below. The function signature, docstring, the `RUBRICS` constant, and all of the surrounding `FastMCP` boilerplate are already filled in for you. You are only writing the body of `get_rubric`.

The function should look up the requested `pitch_type` in the `RUBRICS` dictionary and return the matching entry. If the pitch type is not a key in `RUBRICS`, return `{"name": "Unknown", "criteria": []}`.

After you fill in the function body, run the cell. The cell writes the completed server out to a file called `pitchwise_server.py`, which the agent will launch as a subprocess later in the lab.

Tip: This server has the same shape as the Sticky Notes and Todo servers from the Implement a Model Context Protocol (MCP) Server assignment. The function body is 2 to 3 lines. You can refer back to that assignment if you want to double-check the pattern.

[2]:
```
# Fill in the function body marked YOUR CODE HERE inside the server code below,
# then run the cell to write the server file to disk.

server_code = '''
from mcp.server.fastmcp import FastMCP

# Initialize MCP server
mcp = FastMCP("pitchwise-playbook")

# The Pitchwise partners' rubrics, one per pitch type.
# Each rubric has exactly three criteria the partners care about.
RUBRICS = {
    "consumer": {
        "name": "Consumer Product Pitch Rubric",
        "criteria": [
            "Hook: opens with a specific customer moment, not a category␣
 ↪description.",
            "Why now: explains what changed in the world that makes this␣
 ↪product possible or necessary right now.",
            "Distribution: names a concrete channel and explains why this␣
 ↪product will travel through it.",
        ],
    },
    "b2b_saas": {
        "name": "B2B SaaS Pitch Rubric",
        "criteria": [
            "Customer pain: describes a real, named workflow the buyer hates␣
 ↪today, not a vague 'inefficiency'.",
```

```
            "Market sizing: gives a concrete number of potential buyers and␣
 ↪explains how it was estimated.",
            "Traction: cites a specific signal (a pilot, a paying customer, a␣
 ↪signed letter of intent) instead of generic 'strong interest'.",
        ],
    },
    "deep_tech": {
        "name": "Deep Tech Pitch Rubric",
        "criteria": [
            "Technical edge: explains what the team can do that no one else␣
 ↪can, in language a non-specialist can follow.",
            "Defensibility: names what protects the company from being copied␣
 ↪(patents, proprietary data, specialized talent, hard-to-replicate process)␣
 ↪and for how long.",
            "First customer: identifies a specific first buyer who would pay␣
 ↪for an early version, not a 'future market'.",
        ],
    },
}

@mcp.tool()
def get_rubric(pitch_type: str) -> dict:
    """
    Return the Pitchwise scoring rubric for a given pitch type.

    Args:
        pitch_type: One of 'consumer', 'b2b_saas', or 'deep_tech'.

    Returns:
        dict: A dictionary with 'name' and 'criteria' keys, or
              {'name': 'Unknown', 'criteria': []} if pitch_type is not␣
 ↪recognized.
    """
    # YOUR CODE HERE
    # --- Solution (FastMCP @mcp.tool() body from the Implement a Model Context␣
 ↪Protocol (MCP) Server assignment; function body follows the same shape as␣
 ↪the sticky notes and todo servers in that activity) ---
    if pitch_type in RUBRICS:
        print(f"[MCP] Returning rubric for: {pitch_type}")
        return RUBRICS[pitch_type]
        return {"name": "Unknown", "criteria": []}

    pass

if __name__ == "__main__":
    mcp.run()
```

```
'''

with open("pitchwise_server.py", "w") as f:
    f.write(server_code)

print("Created: pitchwise_server.py")
print("Tools exposed: get_rubric")
```

```
Created: pitchwise_server.py
Tools exposed: get_rubric
```

Now let's confirm your server runs and exposes the `get_rubric` tool before you use it from an agent. The cell below opens a connection to your server with `MCPServerStdio`, calls `list_tools()`, and prints what it finds. This is the same pattern as the `inspect_mcp_server()` cell from the Implement a Model Context Protocol (MCP) Server assignment.

If you see `get_rubric` listed with its `pitch_type` parameter, your server is working.

[3]:
```
# Do not remove or edit this cell

async def inspect_pitchwise_server():
    """Connect to the Pitchwise MCP server and list the tools it exposes."""
    server = MCPServerStdio(
        params={"command": "python", "args": ["pitchwise_server.py"]},
        client_session_timeout_seconds=60
    )

    async with server:
        tools = await server.list_tools()

        print("=" * 60)
        print("PITCHWISE MCP SERVER - Tool Inspection")
        print("=" * 60)
        print(f"\nDiscovered {len(tools)} tool(s):\n")

        for tool in tools:
            print(f"TOOL: {tool.name}")
            print(f"Description: {tool.description}")
            if hasattr(tool, "inputSchema") and tool.inputSchema:
                schema = tool.inputSchema
                if "properties" in schema:
                    print("Parameters:")
                    for param_name, param_info in schema["properties"].items():
                        param_type = param_info.get("type", "unknown")
                        required = param_name in schema.get("required", [])
                        marker = " (required)" if required else " (optional)"
                        print(f"  - {param_name}: {param_type}{marker}")
            print()
```

```
await inspect_pitchwise_server()
```

```
============================================================
PITCHWISE MCP SERVER - Tool Inspection
============================================================

Discovered 1 tool(s):

TOOL: get_rubric
Description:
Return the Pitchwise scoring rubric for a given pitch type.

Args:
    pitch_type: One of 'consumer', 'b2b_saas', or 'deep_tech'.

Returns:
    dict: A dictionary with 'name' and 'criteria' keys, or
          {'name': 'Unknown', 'criteria': []} if pitch_type is not recognized.

Parameters:
  - pitch_type: string (required)
```

1.3 Part 2. Build the Classifier Agent

You now have an MCP server that knows three pitch types: `consumer`, `b2b_saas`, and `deep_tech`. The system needs to recognize which type a draft pitch belongs to so the right rubric and the right coach get used downstream.

This is the routing pattern from the Implement Workflow Routing With OpenAI Agents and LangGraph activity. You will define a Pydantic schema that constrains the classifier's output to those three categories, then build the classifier agent.

Task: Define a Pydantic `BaseModel` called `PitchClassification` with two fields:

1. `pitch_type`: A field whose value must be one of `"consumer"`, `"b2b_saas"`, or `"deep_tech"`. Use `Literal` for this so the value is constrained.
2. `reasoning`: A string explaining why the classifier picked that type.

Tip: This is the same pattern you used in the routing activity for `TicketClassification` and `HelpdeskClassification`. The `Literal` type makes the field's value a fixed set of strings.

[4]:
```
### BEGIN SOLUTION

# --- Solution (Literal-constrained Pydantic BaseModel from the Implement Workflow Routing With OpenAI Agents and LangGraph activity) ---
class PitchClassification(BaseModel):
    """Classification output for a pitch."""
    pitch_type: Literal["consumer", "b2b_saas", "deep_tech"]
```

```
    reasoning: str

### END SOLUTION
```

Task: Define the `classifier_agent`. It should:

1. Be named `"Pitch Classifier"`.
2. Have instructions that tell the model it is classifying a startup pitch into one of the three Pitchwise categories. The instructions should describe each category in a sentence or two so the model knows the difference:
   - `consumer`: A product sold directly to individual people (physical goods, subscription boxes, apps for personal use).
   - `b2b_saas`: Software sold to other businesses on a subscription basis.
   - `deep_tech`: A company whose core advantage comes from a specific piece of novel technology or science (hardware sensors, materials science, robotics, specialized AI infrastructure).
3. Use the `gpt-4.1` model.
4. Use `output_type=PitchClassification` so the agent returns structured output.

[5]:
```
### BEGIN SOLUTION

# --- Solution (Agent with output_type pattern from the OpenAI Agents SDK␣
 ↪Primer and the Implement Workflow Routing With OpenAI Agents and LangGraph␣
 ↪activity) ---
classifier_agent = Agent(
    name="Pitch Classifier",
    instructions="""You are a pitch classifier for Pitchwise, a startup␣
 ↪accelerator.
 Read the pitch and classify it as one of three types:

 - "consumer": A product sold directly to individual people. Examples include␣
 ↪physical goods, subscription boxes, apps for personal use, and␣
 ↪direct-to-consumer brands.
 - "b2b_saas": Software sold to other businesses on a subscription basis. The␣
 ↪buyer is a company, not an individual consumer.
 - "deep_tech": A company whose core advantage comes from a novel technology or␣
 ↪scientific advance. Examples include hardware sensors, materials science,␣
 ↪robotics, and specialized AI infrastructure.

 Output the chosen pitch_type and a one-sentence reasoning that names the␣
 ↪specific signal in the pitch that led you to that category.""",
    model="gpt-4.1",
    output_type=PitchClassification,
)
### END SOLUTION
```

The cell below provides a dictionary that maps each `pitch_type` to a human-readable label you will use when printing results later.

[6]:
```
# Do not remove or edit this cell

PITCH_TYPE_LABELS = {
    "consumer": "Consumer Product",
    "b2b_saas": "B2B SaaS",
    "deep_tech": "Deep Tech",
}
```

Let's see how your classifier handles a couple of example pitches before you build the rest of the system. The cell below runs `classifier_agent` on two short example pitches (one consumer, one B2B SaaS) and prints the result.

If your classifier returns the matching `pitch_type` for each one with sensible reasoning, you are ready to move on.

[7]:
```
# Do not remove or edit this cell

EXAMPLE_PITCH_CONSUMER = "We are MugMail, a monthly subscription that ships a␣
 ↪different small-batch coffee mug to your door each month, paired with the␣
 ↪coffee that inspired it."

EXAMPLE_PITCH_B2B = "We are LedgerLite, a billing tool for solo accountants.␣
 ↪They subscribe and use it to send invoices and track unpaid bills for their␣
 ↪small business clients."

for label, pitch in [("Example A", EXAMPLE_PITCH_CONSUMER), ("Example B",␣
 ↪EXAMPLE_PITCH_B2B)]:
    result = await Runner.run(classifier_agent, pitch)
    classification = result.final_output
    print(f"--- {label} ---")
    print(f"Pitch: {pitch}")
    print(f"Classified as: {classification.pitch_type}␣
 ↪({PITCH_TYPE_LABELS[classification.pitch_type]})")
    print(f"Reasoning: {classification.reasoning}")
    print()
```

```
--- Example A ---
Pitch: We are MugMail, a monthly subscription that ships a different small-batch
coffee mug to your door each month, paired with the coffee that inspired it.
Classified as: consumer (Consumer Product)
Reasoning: The pitch describes a monthly subscription box delivering mugs and
coffee directly to individual consumers, which is characteristic of consumer-
focused products.

--- Example B ---
Pitch: We are LedgerLite, a billing tool for solo accountants. They subscribe
and use it to send invoices and track unpaid bills for their small business
clients.
```

```
Classified as: b2b_saas (B2B SaaS)
Reasoning: The pitch describes a subscription-based software tool sold to solo
accountants for managing client billing, which is a typical B2B SaaS model.
```

1.4 Part 3. Build a Coach Agent

Pitchwise has three coach specialties, one for each pitch type. The coach takes a founder's rough draft and rewrites it so that it addresses the three rubric criteria for that pitch type.

You will build one coach in this part. To save you from repeating the same Agent definition three times, two of the coaches are provided for you. Read them carefully so you understand what good instructions look like for this kind of agent.

[8]:
```
# Do not remove or edit this cell

consumer_coach = Agent(
    name="Consumer Coach",
    instructions="""You are a pitch coach for Pitchwise specializing in␣
 ↪consumer products.
When given a founder's draft pitch, rewrite it so that it does three things␣
 ↪well:
1. Opens with a specific customer moment (not a category description or a␣
 ↪market statistic).
2. Explains why now: what changed in the world that makes this product possible␣
 ↪or necessary today.
3. Names a concrete distribution channel and explains why the product will␣
 ↪travel through it.

Important: do not invent facts that are not in the founder's draft. If the␣
 ↪draft does not contain
the information a criterion needs (for example, no distribution channel is␣
 ↪mentioned), write a
clear placeholder in brackets like [DISTRIBUTION CHANNEL NEEDED: founder must␣
 ↪name a concrete
channel] instead of making one up. It is the founder's job to fill those in,␣
 ↪not yours.

Return only the rewritten pitch. No preamble, no bullet list, no commentary.""",
    model="gpt-4.1",
)

b2b_saas_coach = Agent(
    name="B2B SaaS Coach",
    instructions="""You are a pitch coach for Pitchwise specializing in B2B␣
 ↪SaaS.
When given a founder's draft pitch, rewrite it so that it does three things␣
 ↪well:
```

```
1. Describes a real, named workflow the buyer hates today (not a vague␣
 ↪"inefficiency").
2. Gives a concrete number of potential buyers and explains how that number was␣
 ↪estimated.
3. Cites a specific traction signal (a pilot, a paying customer, a letter of␣
 ↪intent) instead of generic "strong interest."

Important: do not invent facts that are not in the founder's draft. If the␣
 ↪draft does not contain
the information a criterion needs (for example, no traction signal is␣
 ↪mentioned), write a clear
placeholder in brackets like [TRACTION SIGNAL NEEDED: founder must cite a␣
 ↪specific pilot, paying
customer, or letter of intent] instead of making one up. It is the founder's␣
 ↪job to fill those
in, not yours.

Return only the rewritten pitch. No preamble, no bullet list, no commentary.""",
    model="gpt-4.1",
)
```

Task: Define a third coach called `deep_tech_coach` that follows the same pattern as the two coaches above. The deep tech rubric has three criteria, which live in the server file you wrote out in Part 1:

- **Technical edge**: explains what the team can do that no one else can, in language a non-specialist can follow.
- **Defensibility**: names what protects the company from being copied (patents, proprietary data, specialized talent, hard-to-replicate process) and for how long.
- **First customer**: identifies a specific first buyer who would pay for an early version, not a "future market".

Your coach should:

1. Be named `"Deep Tech Coach"`.
2. Use the `gpt-4.1` model.
3. Have instructions that tell the coach it is rewriting a founder's pitch for Pitchwise so that it addresses those three criteria. You can paraphrase the criteria directly into the instructions.
4. Tell the coach not to invent facts that are not in the founder's draft, and to use a bracketed placeholder (for example, `[FIRST CUSTOMER NEEDED: ...]`) instead. This matches the pattern in the two coaches above and prevents the coach from making up customer names, partnerships, or technical claims.
5. Instruct the coach to return only the rewritten pitch, with no preamble.

Tip: You do not need to connect the coach to the MCP server. The coach is generating text from the user's draft; only the evaluator will pull the rubric from MCP later.

[9]:
```
### BEGIN SOLUTION

# --- Solution (Agent definitions with role-specific instructions from the␣
 ↪OpenAI Agents SDK Primer and the Implement an Evaluator-Optimizer Workflow␣
 ↪With OpenAI Agents SDK and LangGraph activity) ---
deep_tech_coach = Agent(
    name="Deep Tech Coach",
    instructions="""You are a pitch coach for Pitchwise specializing in deep␣
 ↪tech.
    When given a founder's draft pitch, rewrite it so that it does three␣
 ↪things well:
    1. Explains the technical edge in language a non-specialist can follow:␣
 ↪what the team can do that no one else can.
    2. Names what protects the company from being copied (patents, proprietary␣
 ↪data, specialized talent, hard-to-replicate process) and for how long.
    3. Identifies a specific first customer who would pay for an early version␣
 ↪of the product, not a "future market."

    Important: do not invent facts that are not in the founder's draft. If the␣
 ↪draft does not contain
    the information a criterion needs (for example, no first customer is␣
 ↪mentioned), write a clear
    placeholder in brackets like [FIRST CUSTOMER NEEDED: founder must name a␣
 ↪specific buyer who
    would pay for an early version] instead of making one up. It is the␣
 ↪founder's job to fill those
    in, not yours.

    Return only the rewritten pitch. No preamble, no bullet list, no␣
 ↪commentary.""",
    model="gpt-4.1",
)
### END SOLUTION
```

Let's see what your `deep_tech_coach` does with a draft pitch. The cell below runs the coach on a short example deep tech pitch (not one of the test pitches you'll see in Part 5) and prints the rewritten output.

[10]:
```
# Do not remove or edit this cell

EXAMPLE_PITCH_DEEP_TECH = "We are FieldOptic, a startup using novel optical␣
 ↪sensors to monitor crop health. Our technology is better than existing␣
 ↪sensors. We believe the agriculture market is huge and we are well␣
 ↪positioned to win."

result = await Runner.run(deep_tech_coach, EXAMPLE_PITCH_DEEP_TECH)
```

```python
print("--- Original draft ---")
print(EXAMPLE_PITCH_DEEP_TECH)
print()
print("--- Rewritten by deep_tech_coach ---")
print(result.final_output)
```

```
--- Original draft ---
We are FieldOptic, a startup using novel optical sensors to monitor crop health.
Our technology is better than existing sensors. We believe the agriculture
market is huge and we are well positioned to win.

--- Rewritten by deep_tech_coach ---
FieldOptic has developed a new kind of optical sensor that can detect subtle
changes in crop health by capturing a broader range of light signals than
current sensors. This allows farmers to identify plant stress and nutrient
deficiencies earlier, supporting better yields. Our edge lies in the unique
sensor design, which is [FOUNDER TO SPECIFY: is it based on custom hardware, a
new algorithm, or both?], making it possible to extract more detailed
information from crops in real time-a capability not available in today's
market.

Our technology is protected by [IP PROTECTION NEEDED: founder must specify if
there are patents filed, trade secrets, or a specialized team], providing us
with a defensible position for [HOW LONG PROTECTED: specify patent life or
exclusivity period].

Our first customer will be [FIRST CUSTOMER NEEDED: founder must name a specific
early-access farm or agribusiness that would pay for initial deployments].
```

1.5 Part 4. Build the Evaluation Pieces

The evaluator-optimizer loop has three agents: a generator, an evaluator, and an optimizer. You built exactly this pattern in the Implement an Evaluator-Optimizer Workflow With OpenAI Agents SDK and LangGraph activity. In this lab, the matched coach plays the role of the generator (it produces the initial draft), and the evaluator and optimizer take over from there.

The evaluator agent needs to call the `get_rubric` MCP tool, which means it needs to be connected to your MCP server. You connect an agent to an MCP server by passing `mcp_servers=[server]` to the `Agent` constructor, as you did in the Implement a Model Context Protocol (MCP) Server assignment. But the server only exists once you open the `async with MCPServerStdio(...) as server:` block. So in this part, you will define everything the evaluator needs except the agent itself: the `PitchEvaluation` schema, and an `EVALUATOR_INSTRUCTIONS` string that the agent will use when it is constructed inside the async block.

Task: Define a Pydantic `BaseModel` called `PitchEvaluation` with three fields:

1. `passes`: A boolean. `True` if the pitch meets all three criteria from the rubric, `False` otherwise.
2. `per_criterion_feedback`: A list of strings. Each string is one sentence of feedback that names which rubric criterion it addresses and what the pitch does well or poorly on that

criterion. The list should have exactly three entries, one per rubric criterion.
3. `summary`: A short one-sentence summary of the verdict.

Tip: This is the same pattern you used in the Implement an Evaluator-Optimizer Workflow With OpenAI Agents SDK and LangGraph activity, but with a list of per-criterion feedback strings instead of a single feedback string. The list structure gives the optimizer something concrete to revise against.

[11]:
```
### BEGIN SOLUTION
# --- Solution (structured evaluation schema from the Implement an␣
 ↪Evaluator-Optimizer Workflow With OpenAI Agents SDK and LangGraph activity;␣
 ↪the list-of-strings field is a small extension that supports per-criterion␣
 ↪feedback) ---
class PitchEvaluation(BaseModel):
    """Structured evaluation output for a pitch."""
    passes: bool
    per_criterion_feedback: list[str]
    summary: str
### END SOLUTION
```

Task: Define an `EVALUATOR_INSTRUCTIONS` string that the evaluator agent will use when it gets constructed in the next cell (and again in Part 5). The string should tell the evaluator to:

1. Call the `get_rubric` tool with the matched pitch type to retrieve the three Pitchwise criteria for this pitch.
2. Score the pitch against each of the three criteria.
3. Return `passes=True` only if all three criteria are clearly met.
4. Return three per-criterion feedback entries, one per rubric criterion, in the same order as the rubric.

Tip: You are writing the prompt text as a Python string. The agent itself gets built inside the `async with` block below (and inside the pipeline function in Part 5) so it can be given the live `server` variable via `mcp_servers=[server]`. This pattern matches the Implement a Model Context Protocol (MCP) Server assignment exactly.

[12]:
```
### BEGIN SOLUTION

# --- Solution (evaluator prompt that directs the agent to call an MCP tool,␣
 ↪combining the structured-feedback evaluator pattern from the Implement an␣
 ↪Evaluator-Optimizer Workflow With OpenAI Agents SDK and LangGraph activity␣
 ↪with the tool-calling guidance pattern from the Implement a Model Context␣
 ↪Protocol (MCP) Server assignment) ---
EVALUATOR_INSTRUCTIONS = """You are an evaluator for Pitchwise. You will be␣
 ↪given a pitch and the pitch type.

 First, call the get_rubric tool with the pitch type to retrieve the three␣
 ↪Pitchwise rubric criteria for this pitch.
```

```
Then evaluate the pitch against each of the three criteria from the rubric. Be␣
↪strict. A criterion is met only if the pitch clearly addresses it, not if␣
↪the pitch is merely close.

Return:
- passes: True only if all three criteria are clearly met; False otherwise.
- per_criterion_feedback: a list of exactly three strings, one per criterion,␣
↪in the same order as the rubric. Each string should name the criterion and␣
↪say what the pitch does well or poorly on it.
- summary: a one-sentence verdict."""

### END SOLUTION
```

The cell below provides the `optimizer_agent` complete. Read it so you understand its role: it takes the current draft and the evaluator's per-criterion feedback, and rewrites the pitch to address every flagged weakness. It is essentially the same agent you wrote in the Implement an Evaluator-Optimizer Workflow With OpenAI Agents SDK and LangGraph activity.

[13]:
```
# Do not remove or edit this cell

optimizer_agent = Agent(
    name="Pitch Optimizer",
    instructions="""You are a pitch revision specialist for Pitchwise. You will␣
↪receive the most recent draft of a pitch and the evaluator's per-criterion␣
↪feedback.

Rewrite the pitch so that it directly addresses every piece of feedback that␣
↪flagged a weakness. Keep the founder's voice and any details that already␣
↪worked.

Important: do not invent facts that are not in the current draft. If the␣
↪evaluator flagged a missing piece of information (for example, no first␣
↪customer is named), do not make one up. Instead, write a clear bracketed␣
↪placeholder like [FIRST CUSTOMER NEEDED: founder must name a specific buyer]␣
↪so the founder knows what to fill in. Sharpening vague language is good.␣
↪Inventing customers, numbers, partnerships, or technical claims is not.

Return only the rewritten pitch. No preamble, no bullet list, no commentary.""",
    model="gpt-4.1",
)
```

Now let's confirm two things at once: that the evaluator can read a rubric from your MCP server, and that it produces the structured feedback shape you defined. The cell below constructs an evaluator agent inside an `async with server:` block (the same pattern you will use in Part 5), runs it on a clean consumer-style example pitch, and prints the structured output.

If you see `passes=True` (or close to it), three per-criterion feedback entries, and a one-sentence summary, your evaluation pieces are wired up correctly.

[14]:
```python
# Do not remove or edit this cell

EXAMPLE_EVAL_PITCH = (
    "On a Saturday morning at the farmers market, a new dog owner picks up a␣
 ↪SniffBox sample "
    "and her puppy chooses the air-dried sweet potato chew over every other␣
 ↪treat on the table. "
    "Pet food labeling rules tightened in California last year, and owners now␣
 ↪want treats with "
    "ingredient lists they can actually read. SniffBox partners with␣
 ↪independent veterinary "
    "clinics, who hand the boxes to first-time puppy owners during their␣
 ↪initial visit, turning "
    "a worried vet appointment into a memorable first treat."
)

async def smoke_test_evaluator():
    server = MCPServerStdio(
        params={"command": "python", "args": ["pitchwise_server.py"]},
        client_session_timeout_seconds=60,
    )

    async with server:
        evaluator_agent = Agent(
            name="Pitch Evaluator",
            instructions=EVALUATOR_INSTRUCTIONS,
            model="gpt-4.1",
            output_type=PitchEvaluation,
            mcp_servers=[server],
        )

        eval_input = f"Pitch type: consumer\n\nPitch:\n{EXAMPLE_EVAL_PITCH}"
        result = await Runner.run(evaluator_agent, eval_input)
        evaluation = result.final_output

        print("--- Evaluator output (structured) ---")
        print(f"passes: {evaluation.passes}")
        print(f"summary: {evaluation.summary}")
        print("per_criterion_feedback:")
        for i, item in enumerate(evaluation.per_criterion_feedback, 1):
            print(f"  {i}. {item}")

await smoke_test_evaluator()
```

```
--- Evaluator output (structured) ---
passes: False
summary: The pitch uses a clear customer hook and describes a concrete
```

```
distribution channel, but it does not fully meet the 'why now' requirement
because the urgency and necessity tied to recent changes are not clearly
established.
per_criterion_feedback:
  1. Hook: The pitch opens with a specific customer moment of a new dog owner
encountering SniffBox at a farmers market, which effectively hooks the audience.
  2. Why now: The pitch references new pet food labeling regulations in
California, but it does not make explicit why these changes make SniffBox
necessary or possible right now; it only loosely connects demand to regulations,
without making a compelling 'why now' case.
  3. Distribution: The pitch concretely states distribution through independent
veterinary clinics and explains the context (first puppy visits), fulfilling
this criterion.
```

1.6 Part 5. Wire the Routing and Evaluator-Optimizer Loop

You now have everything you need: the classifier, the three coaches, the optimizer, the `PitchEvaluation` schema, and the `EVALUATOR_INSTRUCTIONS` string. In this part, you will wire them together into the full pipeline:

1. The classifier picks the pitch type.
2. Inside an `async with MCPServerStdio(...) as server:` block, the evaluator agent is constructed with `mcp_servers=[server]` so it can call the `get_rubric` tool.
3. The matched coach drafts version 1.
4. The evaluator scores it.
5. If the evaluator passes the pitch, the loop ends. Otherwise the optimizer revises and the evaluator scores the new version.
6. The loop runs until the evaluator returns `passes=True` or it hits a hard iteration cap of 3.

Most of the pipeline is already wired for you. The classifier call, the MCP server connection, the evaluator construction inside the async block, the coach dispatch (if/elif/else from the Implement Workflow Routing With OpenAI Agents and LangGraph activity), and the first coach run are all written. Your job is to fill in the loop body.

Task: Complete the body of the `for iteration in range(1, MAX_ITERATIONS + 1):` loop inside `run_pitch_pipeline`. Specifically, on each iteration:

1. Build an evaluator input string that contains both `pitch_type` and `current_draft` so the evaluator knows which rubric to pull. For example, `f"Pitch type: {pitch_type}\n\nPitch:\n{current_draft}"`.
2. Run `evaluator_agent` on that input. Save `.final_output` to a variable called `evaluation`.
3. Print the iteration number and `evaluation.summary` so you can watch the loop converge.
4. If `evaluation.passes` is `True`, break out of the loop.
5. If `iteration < MAX_ITERATIONS`, build an optimizer input from `current_draft` and the joined `per_criterion_feedback`, run `optimizer_agent` on it, and assign the result to `current_draft` so the next iteration evaluates the revised version.

Tip: This is the same loop shape as in the Implement an Evaluator-Optimizer Workflow With OpenAI Agents SDK and LangGraph activity. The only differences are that the evaluator now has an MCP server attached, and routing has already happened above the loop.

[15]:
```
# Do not remove or edit this cell

MAX_ITERATIONS = 3
```

[16]:
```
async def run_pitch_pipeline(pitch_id: str, pitch_text: str) -> dict:
    """
    Run a draft pitch through the full Pitchwise pipeline:
    classify -> coach -> evaluator-optimizer loop -> return final pitch.
    """

    # Step 1: classify the pitch (provided)
    classification_result = await Runner.run(classifier_agent, pitch_text)
    classification = classification_result.final_output
    pitch_type = classification.pitch_type
    print(f"\n=== {pitch_id}: classified as {PITCH_TYPE_LABELS[pitch_type]}␣
 ↪===")
    print(f"Reasoning: {classification.reasoning}\n")

    # Step 2: connect to the MCP server (provided)
    server = MCPServerStdio(
        params={"command": "python", "args": ["pitchwise_server.py"]},
        client_session_timeout_seconds=60,
    )

    async with server:
        # Construct the evaluator agent with the MCP server attached (provided)
        evaluator_agent = Agent(
            name="Pitch Evaluator",
            instructions=EVALUATOR_INSTRUCTIONS,
            model="gpt-4.1",
            output_type=PitchEvaluation,
            mcp_servers=[server],
        )

        # Coach dispatch (provided): pick the matched coach for this pitch type
        if pitch_type == "consumer":
            coach = consumer_coach
        elif pitch_type == "b2b_saas":
            coach = b2b_saas_coach
        else:
            coach = deep_tech_coach

        # First coach run (provided): produce the initial draft
        coach_result = await Runner.run(coach, pitch_text)
        current_draft = coach_result.final_output

        evaluation = None
```

```
        # Do not remove or edit this line: the iteration cap protects against␣
↪runaway loops.
        for iteration in range(1, MAX_ITERATIONS + 1):
            ### BEGIN SOLUTION
            # --- Solution (loop pattern from the Implement an␣
↪Evaluator-Optimizer Workflow With OpenAI Agents SDK and LangGraph activity)␣
↪---
            eval_input = f"Pitch type: {pitch_type}\n\nPitch:\n{current_draft}"
            eval_result = await Runner.run(evaluator_agent, eval_input)
            evaluation = eval_result.final_output
            print(f"Iteration {iteration}: {evaluation.summary}")

            if evaluation.passes:
                break

            if iteration < MAX_ITERATIONS:
                feedback = "\n".join(f"- {item}" for item in evaluation.
↪per_criterion_feedback)
                opt_input = f"Current pitch:\n{current_draft}\n\nFeedback to␣
↪address:\n{feedback}"
                opt_result = await Runner.run(optimizer_agent, opt_input)
                current_draft = opt_result.final_output
            ### END SOLUTION

            pass

    return {
        "pitch_type": pitch_type,
        "final_draft": current_draft,
        "evaluation": evaluation,
        "iterations": iteration,
    }
```

Now let's see the full pipeline in action on three real founder pitches. Three founders in the current cohort have sent in drafts this week. The Pitchwise partners have asked you to run them through your pipeline so they can see how the system behaves.

The three pitches are deliberately different from each other:

- **Pitch A** is a clean consumer pitch with solid material on all three rubric criteria.
- **Pitch B** is a B2B SaaS pitch where one of the three criteria is stated vaguely and should sharpen on revision.
- **Pitch C** is a deep tech pitch that is missing or buzzword-thin on the criteria the rubric will care about.

[17]:
```
# Do not remove or edit this cell
```

```
PITCH_A = """We are SnackPocket, a monthly snack subscription box for kids ages␣
 ↪6 to 12. Last Tuesday a mom in our pilot group texted us a photo of her␣
 ↪seven-year-old reading the SnackPocket ingredient label out loud at the␣
 ↪breakfast table, sounding out each word. That moment is why parents keep␣
 ↪telling us they want better snacks for their kids but get exhausted reading␣
 ↪every label themselves at the grocery store. Three big school districts in␣
 ↪our region tightened their snack guidelines last year, so the bar for what␣
 ↪kids are allowed to bring to lunch is suddenly higher and parents are␣
 ↪scrambling. We curate boxes built around real-food ingredients and ship␣
 ↪monthly with a kid-friendly trading card inside each box. We are launching␣
 ↪through three school PTA fundraisers next month, which already reach␣
 ↪hundreds of families per school and have a built-in referral hook for␣
 ↪parents."""

PITCH_B = """ShiftRoster is software that helps small businesses manage␣
 ↪employee schedules. Owners of small restaurants and retail shops spend three␣
 ↪to five hours every Sunday night rebuilding next week's shift schedule in␣
 ↪spreadsheets, juggling time-off requests, double-booked employees, and␣
 ↪last-minute swaps. Our product lets them generate a full week's schedule in␣
 ↪under five minutes by dragging and dropping employees onto a calendar. There␣
 ↪are roughly 700,000 small restaurants and retail businesses in the US with␣
 ↪five to fifty hourly employees, which is the band where spreadsheet␣
 ↪scheduling breaks down but full enterprise tools are overkill. We have␣
 ↪strong interest from local restaurants and retail shops in our city and plan␣
 ↪to launch in the next quarter."""

PITCH_C = """KitchenSense is a next-generation IoT platform leveraging AI to␣
 ↪revolutionize restaurant operations. Using our proprietary sensor technology␣
 ↪and machine learning algorithms, we deliver actionable insights that drive␣
 ↪operational excellence across the food service vertical. With the rapid␣
 ↪digital transformation of the restaurant industry, our addressable market is␣
 ↪massive. We are uniquely positioned to capture significant share."""
```

The cell below runs your pipeline on all three pitches in sequence. Run it and watch what happens. The cell prints the classification, the evaluator's verdict at each iteration, and the final draft for each pitch.

Note: Running all three pitches will make several API calls per pitch, so the cell may take 30 to 60 seconds to complete. You will be able to see the loop progress because each iteration prints as it happens.

[18]:
```
# Do not remove or edit this cell

results = {}

for pitch_id, pitch_text in [("pitch_a", PITCH_A), ("pitch_b", PITCH_B),␣
 ↪("pitch_c", PITCH_C)]:
    result = await run_pitch_pipeline(pitch_id, pitch_text)
```

```
    results[pitch_id] = result
    print(f"\n--- {pitch_id} FINAL ({result['iterations']} iteration(s),␣
↪passed={result['evaluation'].passes}) ---")
    display(Markdown(f"**Final draft:**\n\n{result['final_draft']}"))
    print(f"\nFinal verdict: {result['evaluation'].summary}")
    print(f"\nPer-criterion feedback from the last evaluation:")
    for i, item in enumerate(result['evaluation'].per_criterion_feedback, 1):
        print(f"  {i}. {item}")
    print("=" * 70)
```

```
=== pitch_a: classified as Consumer Product ===
Reasoning: The pitch describes a monthly subscription box of snacks sold
directly to parents for their children, which is a direct-to-consumer model.

Iteration 1: This consumer pitch effectively meets all three Pitchwise criteria
with a clear hook, strong 'why now', and a specific, well-justified distribution
strategy.

--- pitch_a FINAL (1 iteration(s), passed=True) ---
```

Final draft:

Last Tuesday, a mom in our pilot group texted us a photo of her seven-year-old reading the SnackPocket ingredient label out loud at the breakfast table, sounding out each word. It's moments like these that show us how parents want better snacks for their kids but are exhausted from having to read every label at the grocery store themselves. Last year, three big school districts in our region tightened their snack guidelines, raising the bar for what kids are allowed to bring to lunch and leaving parents scrambling for compliant, healthy options. We curate monthly snack boxes built around real-food ingredients, each with a kid-friendly trading card inside, making it easy and fun for families to choose better snacks. We're launching through three school PTA fundraisers next month; these already reach hundreds of families per school and offer a built-in referral hook for parents, giving us immediate, trusted access to our target market.

```
Final verdict: This consumer pitch effectively meets all three Pitchwise
criteria with a clear hook, strong 'why now', and a specific, well-justified
distribution strategy.

Per-criterion feedback from the last evaluation:
  1. Hook: The pitch opens with a vivid, specific moment of a parent watching
her child engage with the product, drawing immediate emotional
resonance-criterion clearly met.
  2. Why now: The pitch clearly explains that recent changes to school snack
guidelines have heightened the challenge for parents, providing timely
relevance-criterion clearly met.
  3. Distribution: The pitch concretely identifies immediate access through
school PTA fundraisers, with a rationale for why this channel offers trusted
access to the target market-criterion clearly met.
```

```
======================================================================

=== pitch_b: classified as B2B SaaS ===
Reasoning: The pitch describes software sold to businesses (specifically small
restaurants and retail shops) to help manage employee scheduling, which is a
classic example of business-to-business SaaS.

Iteration 1: The pitch clearly describes the pain point and market size, but
fails the traction criterion because no specific signal is given.
Iteration 2: The pitch is strong on pain and market sizing but does not meet the
traction criterion, so it does not fully pass.
Iteration 3: The pitch fails because it does not provide a concrete traction
signal, which is required to pass all criteria.

--- pitch_b FINAL (3 iteration(s), passed=False) ---
```

Final draft:

Every Sunday night, owners of small restaurants and retail shops spend three to five frustrating hours using spreadsheets to manually rebuild the next week's shift schedule. They're forced to sift through email threads for last-minute time-off requests, check paper notes to avoid double-booking employees, and handle chaotic text chains when workers swap shifts at the last minute. ShiftRoster lets these owners generate a full week's schedule in under five minutes by simply dragging and dropping employees onto a digital calendar, with built-in conflict checks for time-off and shift swaps.

There are approximately 700,000 small restaurants and retail businesses in the US with five to fifty hourly employees, based on National Restaurant Association and U.S. Census Bureau data. This is the segment where manual spreadsheets are still common, but more expensive enterprise HR tools are not a fit.

[FIRST CUSTOMER NEEDED: founder must name a specific pilot, paying customer, or letter of intent]

```
Final verdict: The pitch fails because it does not provide a concrete traction
signal, which is required to pass all criteria.

Per-criterion feedback from the last evaluation:
  1. Customer pain: The pitch clearly describes a real workflow pain faced by
restaurant and retail shop owners, detailing their frustration with manual
scheduling using spreadsheets, emails, notes, and texts.
  2. Market sizing: The pitch successfully provides a concrete number of
potential buyers and specifies the data sources and segment to justify the
figure.
  3. Traction: The pitch lacks a specific customer signal such as a pilot,
paying customer, or letter of intent; it includes a placeholder indicating this
is missing.
======================================================================
```

```
=== pitch_c: classified as Deep Tech ===
Reasoning: The pitch specifically mentions proprietary sensor technology and
machine learning algorithms as its core, signaling a foundation in novel
technology characteristic of deep tech.

Iteration 1: The pitch is clear about its technical edge but fails to meet the
bar on defensibility and naming a specific first customer.
Iteration 2: While the technical edge is well-articulated, KitchenSense's pitch
falls short on clearly explaining defensibility mechanisms and fails to name a
specific first customer.
Iteration 3: The pitch is strong on technical edge but fails on both
defensibility details and naming a first customer, so it does not pass the
rubric.

--- pitch_c FINAL (3 iteration(s), passed=False) ---
```

Final draft:

KitchenSense's unique technical edge lies in its proprietary sensors, purpose-built to capture real-time data from kitchen equipment and restaurant environments, paired with machine learning algorithms that analyze this data to recommend actionable improvements for restaurant operations. Unlike generic IoT solutions, our system is specifically tailored to the fast-paced, variable demands of food service, delivering insights that off-the-shelf platforms cannot provide.

Our defensibility relies on our proprietary sensor technology, unique data sets collected from actual restaurant environments, and in-house developed algorithms created by a team with deep expertise in both AI and food service operations. We are [DETAILS NEEDED: founder must specify whether patents have been filed, are pending, or are planned for our sensors or algorithms], and we protect our algorithms and data through [DETAILS NEEDED: founder must clarify if trade secrets, proprietary data rights, or other legal/commercial barriers are in place, and specify any time horizons or terms relevant to these protections]. These elements form significant barriers to replication, making it difficult for competitors to match our solution.

Our go-to-market strategy begins with a signed pilot agreement with [FIRST CUSTOMER NEEDED: founder must name a specific restaurant or food service operator ready to implement and pay for an early version], which will secure immediate validation and initial revenue as we launch.

```
Final verdict: The pitch is strong on technical edge but fails on both
defensibility details and naming a first customer, so it does not pass the
rubric.

Per-criterion feedback from the last evaluation:
  1. Technical edge: The pitch clearly explains the unique technical edge,
detailing proprietary sensors and tailored machine learning for restaurant
operations in accessible language.
  2. Defensibility: The pitch lists possible defensibility avenues (sensors,
data, algorithms, team expertise) but fails to specify whether patents have been
filed or what concrete protections are in place, and lacks any mention of
```

```
timeframes.
  3. First customer: The pitch does not identify any specific first buyer, only
referencing a future need for a named pilot customer.
======================================================================
```

Task: Review the three runs above. In the markdown cell below, answer the following:

1. How many iterations did each pitch take? Which pitches converged (the evaluator returned `passes=True`) and which hit the iteration cap without converging?
2. For the pitch that took the most iterations (or hit the cap), pick one of the three rubric criteria and quote a sentence from the evaluator's per-criterion feedback that explains why the pitch struggled on that criterion. Did the optimizer eventually fix it, or did the same weakness keep showing up across iterations?
3. The evaluator agent calls the `get_rubric` MCP tool every time it runs. Why is that better than putting the rubric directly into the evaluator's instructions? Name one concrete benefit and one concrete drawback.

<Double click this Markdown cell to make it editable, and record your findings here.>

1.7 Part 6. Analysis

You have now built a complete agentic pitch coaching system for Pitchwise, combining routing, evaluator-optimizer, and MCP into one pipeline. In this section, reflect on the system as a whole.

Answer the following questions in the markdown cell below:

1. **The pitch that did not converge**: Look at the pitch that hit the iteration cap (or, if all three converged, one of the pitches that took the most iterations). Quote one specific line from its final draft and explain what the evaluator was still unhappy about. What does this tell you about the limits of what an evaluator-optimizer loop can fix? Is there a kind of initial pitch where adding more iterations would obviously not help?

2. **Explaining the system to a Pitchwise partner**: One of the Pitchwise partners is not technical and wants to know, in plain language, why you built it the way you did. They specifically want to understand why the evaluator pulls the rubric from an MCP server instead of just having the rubric "baked in" to the evaluator's instructions. Write a short explanation (3 to 5 sentences) you could give the partner. Avoid using complex, technical jargon. Do not assume they know what MCP, a rubric tool, or an "agent" is.

<Double click this Markdown cell to make it editable, and record your findings here.>

1.8 Part 7. Reflection: AI Usage

1. Did you use AI tools for this lab? If yes, which ones and at what points in your work? If no, briefly explain your reasoning.
2. If you used AI, describe one specific prompt that was useful and explain why it worked. If you did not use AI, walk through one part of the lab where you had to figure something out on your own and explain how you got there.
3. How did you verify that your work was correct? What would you look for to catch a mistake, whether it came from AI or from your own reasoning?
4. What is one thing you would do differently next time, either in how you approached the lab or in how you used (or did not use) AI?

Record your findings in the cell below.

<Double click this Markdown cell to make it editable, and record your findings here.>